

KompozeR

Applicazioni e Servizi Web

Valerio Giannini - 0001125005 {valerio.giannini@studio.unibo.it}

08 Luglio 2026

0.1 Introduzione

KompozeR è una web application che estende le funzionalità dell'e-commerce di Kompo (www.soluzionekompo.com), azienda specializzata in mobili componibili personalizzabili. L'applicazione permette all'utente di progettare la propria scaffalatura tramite un configuratore visuale interattivo, ottenendo un'anteprima del risultato finale e popolando automaticamente il carrello con i componenti selezionati. L'utente oltre alla creazione del proprio scaffale personalizzato potrà accedere a funzionalità avanzate di assistenza e monitoraggio in tempo reale. Un chatbot assistente in tempo reale permetterà all'utente di ricevere supporto per informazioni sui prodotti del catalogo. Il sistema notifica inoltre l'utente in tempo reale in caso di variazioni di disponibilità o prezzo dei componenti inseriti nella propria configurazione. Lato amministratore, KompozeR fornisce strumenti per la gestione del catalogo prodotti, un sistema di monitoraggio degli ordini (con aggiornamento dello stato operativo dell'ordine) e una pagina di reportistica per osservare i trend.

0.1.1 Contesto

Kompo è un'azienda che si occupa di mobili componibili personalizzabili, offrendo soluzioni su misura per ogni esigenza abitativa. Il catalogo si divide in 3 categorie di prodotti: Tondo, Quadro e Kube, non compatibili tra loro ma con caratteristiche e funzionalità simili. Questo aspetto è rilevante perché la logica di composizione dei componenti viene poi mantenuta omogenea all'interno della singola categoria. Il processo di acquisto attuale prevede che l'utente selezioni i singoli componenti desiderati, avendo già un'idea chiara del risultato finale, e li aggiunga al carrello. Questo processo può risultare complesso e poco intuitivo per gli utenti che non conoscono il prodotto e si trovano costretti a chiedere assistenza direttamente al personale di vendita, con conseguente aumento dei tempi di acquisto e potenziale insoddisfazione del cliente. KompozeR nasce con l'obiettivo di semplificare e migliorare l'esperienza di acquisto, offrendo un configuratore visuale il più semplice e intuitivo possibile, garantendo a tutti la possibilità di acquisto.

0.2 Requisiti

La definizione dei requisiti di KompozeR e' stata costruita seguendo un approccio orientato all'utente. In particolare, la fase di analisi e' stata articolata a partire dall'individuazione degli stakeholder principali, dalla costruzione di personas rappresentative e dalla descrizione di scenari e casi d'uso centrali. Questo consente di giustificare i requisiti di business, funzionali e non funzionali come derivazione diretta di esigenze concrete del dominio applicativo.

0.2.1 Stakeholder

L'identificazione degli stakeholder consente di chiarire chi trae valore dal sistema, chi lo utilizza direttamente e chi influenza i vincoli progettuali.

Stakeholder primari

Stakeholder	Descrizione	Obiettivi principali	Bisogni critici
Cliente finale	Utente che desidera progettare e acquistare una scaffalatura personalizzata	Configurare rapidamente il prodotto corretto, capire prezzo e compatibilita', completare l'acquisto senza errori	Configuratore chiaro, anteprima affidabile, salvataggio configurazioni
Cliente registrato ricorrente	Utente che torna sul sito per modificare o riordinare una configurazione	Riordinare o modificare configurazioni salvate	Storico configurazioni, accesso rapido
Amministratore catalogo	Operatore che gestisce prodotti, componenti, disponibilita' e prezzi	Mantenere coerente il catalogo e rendere acquistabili solo combinazioni valide	Interfacce di gestione chiare, aggiornamenti rapidi, controllo consistenza dati

Stakeholder secondari

Stakeholder	Ruolo nel progetto	Interesse principale
Azienda Kompo	Committente del prodotto	Aumentare vendite, ridurre abbandono in fase di progettazione, migliorare esperienza d'acquisto
Team di sviluppo	Realizza e mantiene il sistema	Requisiti chiari, comportamento verificabile, architettura manutenibile ed estendibile
Servizio clienti	Supporta utenti con dubbi o problemi	Ridurre richieste ripetitive grazie a configuratore e assistenza automatizzata
Responsabile business/reportistica	Analizza andamento ordini e utilizzo del sistema	Ottenere insight utili su trend ordini

0.2.2 Personas

Per guidare la definizione dei requisiti sono state individuate tre personas rappresentative.

Marta, cliente orientata alla praticita'

Marta ha 34 anni, e' architetta freelance e desidera arredare uno studio domestico con una soluzione modulare compatibile con lo spazio disponibile. Ha buone competenze digitali e si aspetta un configuratore guidato, capace di mostrare il risultato finale e il prezzo complessivo. Le criticità che solitamente incontra riguardano la complessità dei cataloghi tecnici, i dubbi sulla compatibilità tra componenti, un mancato check sulla correttezza del progetto e il rischio di perdere il lavoro svolto durante la configurazione.

Luca, cliente indeciso ma motivato all'acquisto

Luca ha 28 anni, e' impiegato e vuole arredare il soggiorno pur non conoscendo in dettaglio il dominio Kompo. Ha competenze digitali medie e necessita di un'interfaccia semplice, di un supporto contestuale sui prezzi e della possibilità di mettere in standby una configurazione per riprenderla in seguito. La sua esigenza principale e' evitare errori di comprensione durante il processo di acquisto.

Fabio, amministratore del catalogo

Fabio ha 48 anni ed e' responsabile operativo dell'e-commerce. Si occupa di aggiornare prezzi e disponibilità dei componenti e ha bisogno di strumenti amministrativi chiari, rapidi e coerenti con il resto del sistema. Il suo obiettivo e' garantire che le configurazioni costruite dagli utenti restino allineate con i dati reali del catalogo.

0.2.3 Scenari d'uso principali

- **Configurazione iniziale:** l'utente seleziona una categoria di prodotto, inserisce eventuali preferenze iniziali e compone la scaffalatura attraverso i comandi del configuratore visuale. Il sistema mostra soltanto componenti compatibili e aggiorna dinamicamente anteprima e prezzo.
- **Supporto contestuale:** durante la configurazione l'utente apre il chatbot e richiede chiarimenti sul prezzo di uno o più componenti, senza interrompere il flusso operativo principale.
- **Salvataggio e ripresa:** l'utente autenticato salva una configurazione (automaticamente dopo ogni step), la recupera in una sessione successiva per terminarla o modificarla.
- **Notifica di variazione:** il sistema informa l'utente quando una configurazione salvata o attiva viene impattata da una variazione di prezzo o disponibilità di uno dei componenti coinvolti.
- **Gestione amministrativa:** l'amministratore aggiorna dati di catalogo e consulta ordini e report sintetici per monitorare l'andamento commerciale.

0.2.4 Casi d'uso principali

UC1 – Configurazione di una scaffalatura

Attore principale Cliente finale.

Precondizioni Il configuratore è disponibile e il sistema dispone dei componenti necessari alla composizione.

Flusso principale L'utente avvia il configuratore, inserisce eventuali vincoli iniziali, seleziona o conferma la categoria di prodotto, modifica la configurazione e visualizza gli aggiornamenti dinamici di anteprima e prezzo. Il sistema consente esclusivamente l'uso di componenti compatibili con la categoria scelta.

Postcondizioni Esiste una configurazione coerente, pronta per essere salvata o aggiunta al carrello.

UC2 – Richiesta di assistenza contestuale

Attore principale Cliente finale.

Precondizioni L'utente si trova in una fase di configurazione e il chatbot è disponibile.

Flusso principale L'utente apre il chatbot, formula una domanda relativa ai componenti e riceve una risposta contestuale che gli consente di proseguire la configurazione.

Postcondizioni L'utente ottiene supporto senza uscire dal configuratore.

UC3 – Salvataggio e ripresa di una configurazione

Attore principale Utente autenticato.

Precondizioni L'utente ha creato almeno una configurazione ed e' autenticato.

Flusso principale L'utente crea e aggiorna progressivamente la configurazione nel CAD, la ritrova nel proprio profilo in una sessione successiva e la riapre per modificarla o completarla.

Postcondizioni La configurazione risulta persistita e riutilizzabile in sessioni future.

UC4 – Notifica di variazione prezzo o disponibilita'

Attore principale Utente autenticato.

Attore secondario Amministratore catalogo.

Precondizioni Esiste almeno un contesto utente impattabile (configurazione finalizzata, carrello o sottoscrizione prodotto) e uno dei componenti associati subisce una variazione di prezzo o disponibilita'.

Flusso principale L'amministratore aggiorna il dato di catalogo, il sistema propaga l'evento di variazione e invia una notifica agli utenti coinvolti tramite i contesti monitorati.

Postcondizioni L'utente e' informato delle variazioni che possono influenzare la propria configurazione.

UC5 – Gestione amministrativa e reportistica

Attore principale Amministratore.

Precondizioni L'amministratore e' autenticato nell'area gestionale.

Flusso principale L'amministratore aggiorna prezzi e disponibilita' dei componenti, consulta gli ordini e accede a una sezione di reportistica sintetica basata su trend ordini.

Postcondizioni I dati risultano aggiornati e l'amministratore dispone di informazioni utili a monitorare l'andamento delle vendite.

0.2.5 Requisiti di business

BR1 Il sistema deve supportare la vendita di scaffalature personalizzabili semplificando il processo di composizione rispetto alla procedura di acquisto tradizionale.

BR2 Il sistema deve aumentare il valore percepito dell'e-commerce Kompo offrendo un configuratore visuale e servizi di assistenza digitale integrati.

- BR3** Il sistema deve ridurre il rischio di errori d'acquisto dovuti a scarsa comprensione di prezzi, disponibilit , compatibilit  tra componenti e stato della configurazione.
- BR4** Il sistema deve consentire all'azienda di mantenere allineati configurazioni utente, dati di catalogo e informazioni operative.
- BR5** Il sistema deve fornire all'area amministrativa una vista sintetica utile al monitoraggio di ordini e trend ordini.

0.2.6 Requisiti funzionali

- FR1** Il sistema deve fornire un configuratore visuale per la composizione della scaffalatura all'interno della categoria di prodotto selezionata.
- FR2** Il sistema deve aggiornare l'anteprima della scaffalatura in seguito alle modifiche effettuate nel configuratore.
- FR3** Il sistema deve aggiornare il prezzo stimato della configurazione in seguito alle modifiche effettuate nel configuratore.
- FR4** Il sistema deve consentire di aggiungere al carrello una configurazione completata.
- FR5** Il sistema deve fornire un chatbot integrato per rispondere a richieste riguardanti il catalogo.
- FR6** Il sistema deve consentire agli utenti autenticati di salvare una configurazione associandola al proprio profilo durante il flusso di configurazione.
- FR7** Il sistema deve consentire agli utenti autenticati di visualizzare e riprendere configurazioni salvate in precedenza.
- FR8** Il sistema deve notificare agli utenti autenticati variazioni di prezzo o disponibilit  che impattano contesti rilevanti (configurazioni finalizzate, carrello o sottoscrizioni prodotto).
- FR9** Il sistema deve impedire la composizione di scaffalature che mescolano componenti appartenenti a categorie di prodotto diverse.
- FR10** Il sistema deve consentire agli amministratori di aggiornare dati di catalogo, inclusi prezzo e disponibilit  dei componenti.
- FR11** Il sistema deve rendere disponibili agli amministratori una vista ordini e una sezione di reportistica sintetica sui trend ordini.

0.2.7 Requisiti non funzionali

- NFR1** L'interfaccia del configuratore deve permettere a un utente con competenze digitali medie di comprendere le azioni principali senza formazione preventiva.
- NFR2** Il sistema deve mostrare feedback immediato a seguito delle azioni rilevanti dell'utente, in particolare durante configurazione, salvataggio e aggiornamento prezzo.
- NFR3** Le principali funzionalita' del sistema devono essere progettate in modo coerente con i principi base di accessibilita' web e con l'uso corretto di elementi semantici.
- NFR4** L'interfaccia deve risultare fruibile sia da dispositivi desktop sia da dispositivi mobili, preservando l'accesso alle funzionalita' essenziali.
- NFR5** L'aggiornamento di anteprima e prezzo nel configuratore deve essere percepito dall'utente come tempestivo durante l'interazione.
- NFR6** Le notifiche di variazione prezzo o disponibilita' devono essere recapitate in modo consistente agli utenti interessati quando il contesto applicativo lo consente.
- NFR7** Le funzionalita' amministrative devono essere accessibili solo ad utenti autenticati con ruolo autorizzato.
- NFR8** Il sistema deve memorizzare soltanto i dati necessari alla gestione di account, configurazioni salvate e operazioni applicative previste.
- NFR9** Il sistema deve mantenere separati i moduli relativi a configurazione, gestione catalogo, notifiche e reportistica in modo da facilitare evoluzione e manutenzione.
- NFR10** Il sistema deve evitare funzionalita' superflue e privilegiare interazioni essenziali, riducendo complessita', traffico e carico computazionale non necessario.

0.2.8 Tracciabilita'

I requisiti presentati in questa sezione derivano dall'analisi degli stakeholder, dalle personas individuate e dagli scenari e casi d'uso principali. Questa struttura garantisce la coerenza tra bisogni degli utenti, comportamento atteso del sistema e qualita' richieste al prodotto finale.

0.3 Design

Questa sezione descrive la progettazione architettonica di KompozeR in termini di strutturazione del dominio, separazione delle responsabilità e coerenza tra modello e implementazione. L'obiettivo è mostrare come requisiti, bounded context, servizi, contratti applicativi e persistenza siano stati allineati in un'unica soluzione tecnica.

Nel seguito si fa riferimento anche agli artefatti di progettazione aggiornati in *utilities* (bounded context, mappa endpoint, definizione DB e domain DTO), riallineati rispetto allo stato corrente del codice.

0.3.1 Obiettivi di design

Le principali decisioni progettuali sono state prese con l'intento di raggiungere i seguenti obiettivi:

- separazione chiara tra frontend, gateway e servizi backend;
- indipendenza dei moduli del dominio (autenticazione, catalogo, configurazione CAD, carrello, ordini, notifiche, chatbot, reporting);
- favorire manutenibilità, evoluzione incrementale e chiarezza delle responsabilità;
- supportare un'esperienza utente interattiva, con feedback rapidi su configurazione, prezzo e notifiche;
- centralizzare autenticazione e controllo accessi nel gateway, riducendo duplicazione di logica nei microservizi.

0.3.2 Visione architettonica

KompozeR è stato progettato come una Single Page Application che comunica con un backend organizzato in più microservizi con architettura esagonale che aderisce al modello REST (Representational State Transfer). Il frontend concentra la logica di presentazione e di interazione con l'utente, mentre il backend espone servizi applicativi separati per area di responsabilità.

Dal punto di vista logico, l'architettura si articola in tre livelli:

- frontend web, responsabile dell'interazione con utenti finali e amministratori;
- API Gateway, punto di ingresso unico per richieste HTTP e handshake real-time;
- microservizi backend, ciascuno focalizzato su un sottoinsieme specifico del dominio.

0.3.3 Suddivisione in microservizi

Bounded Context

1. Authentication (support)

- Responsabilita: gestione identita, login, registrazione, sessioni e ruoli.
- Possiede: utenti (**users**) e sessioni (**sessions**).
- Non possiede: catalogo, configurazioni, carrello, ordini, notifiche, chat, reporting.

2. Catalog (support)

- Responsabilita: gestione componenti, prezzo, disponibilita e ricerca catalogo.
- Possiede: componenti (**components**).
- Pubblica eventi su Redis (**catalog:events**) quando cambiano prezzo/disponibilita.

3. CAD (core)

- Responsabilita: ciclo di vita della configurazione scaffalatura.
- Possiede: configurazioni (**configurations**) con stato, ambiente, categoria, piano colonne, design e BOM.
- Dipende da Catalog per regole/componenti e da Cart/Notification per finalizzazione e sottoscrizioni.

4. Cart (core)

- Responsabilita: gestione carrello utente e checkout.
- Possiede: carrelli (**carts**) con item snapshot (sku, prezzo unitario, quantita, totale riga).
- Reagisce agli eventi catalogo per rimuovere/ripristinare articoli e riallineare prezzi.

5. Order (core)

- Responsabilita: persistenza ordini e transizioni di stato.
- Possiede: ordini (**orders**) con stati SUBMITTED, DONE, CANCELLED.

6. Notification (core)

- Responsabilita: sottoscrizioni utente e notifiche in-app.
- Possiede: notifiche (**notifications**), sottoscrizioni (**notificationSubscriptions**), eventi processati (**notificationEvents**).
- Consuma eventi catalogo da Redis e risolve impatti su contesti CAD/CART/SUBSCRIPTION.

7. Chatbot (core)

- Responsabilita: sessioni chat e messaggistica contestuale.
- Possiede: sessioni (`chatSessions`) e messaggi (`chatMessages`).
- Dipende da Catalog e CAD per risposte contestuali.

8. Reporting (support)

- Responsabilita: trend ordini per area amministrativa.
- Non possiede un DB dedicato nel setup corrente: legge la collection `orders` (`orderdb`) in sola lettura.

Context Map (stato corrente)

- Frontend → API Gateway: entry point unico per API e websocket.
- API Gateway → Authentication: autenticazione/ruoli/sessioni.
- API Gateway → Catalog/CAD/Cart/Order/Notification/Chatbot/Reporting: proxy REST.
- Catalog → Notification: eventi `PRICE_CHANGED` e `AVAILABILITY_CHANGED` su Redis.
- Catalog → Cart: eventi Redis per riallineamento carrello.
- CAD → Cart: finalizzazione configurazione e sincronizzazione BOM.
- CAD → Notification: creazione sottoscrizioni prodotto in fase di finalze.
- Order → Reporting: reporting aggrega trend da `orders`.
- Chatbot → Catalog/CAD: recupero contesto per risposte.

Entita globali principali

Entita	Owner	Uso principale
User	Authentication	Identita e ruolo (GUEST, BASE, ADMIN)
Component	Catalog	SKU, categoria, tipo, prezzo, disponibilita
Configuration	CAD	Workflow configurazione + BOM
Cart	Cart	Carrello utente e checkout
Order	Order	Stato ordine e storico operativo
Notification	Notification	Notifiche prezzo/disponibilita
NotificationSubscription	Notification	Monitoraggio SKU per utente
ChatSession / ChatMessage	Chatbot	Assistenza contestuale

0.3.4 API Gateway e comunicazione tra componenti

Il gateway semplifica la comunicazione client-backend, verifica JWT e controllo accessi. Il frontend interagisce con un unico endpoint e non dipende da URL o policy specifiche dei singoli servizi.

Il flusso e' il seguente:

- il client invia **Authorization: Bearer <token>** al gateway;
- il gateway valida il token e propaga ai servizi header di identita' (es. **X-User-Id**, **X-User-Role**, **X-Session-Id**);
- il routing proxy inoltra la richiesta al servizio corretto (auth, catalog, cad, cart, orders, notifications, chatbot, reports).

Le interazioni principali usano REST. Le funzionalita' real-time usano Socket.IO:

- canale notifiche in-app per push di eventi utente;
- canale chatbot per invio/ricezione messaggi in tempo quasi reale.

Le notifiche dominio nascono da eventi catalogo (prezzo/disponibilita') pubblicati su Redis e consumati dal notificationService.

0.3.5 Modello del dominio

Dal punto di vista applicativo, il dominio ruota attorno a entita' centrali: utente, prodotto, configurazione, carrello, ordine, sessione chat, notifica. Le relazioni principali sono:

- una **Configuration** appartiene a un utente ed e' vincolata a una categoria (TONDO, QUADRO, KUBE);
- la finalizzazione di una configurazione genera una BOM e aggiorna il carrello;
- il **Catalog** e' fonte autorevole di prezzo/disponibilita';
- cambi catalogo possono impattare i contesti utente (configurazioni finalizzate, carrelli, sottoscrizioni) e generare notifiche;
- il **Reporting** aggrega i dati ordine per intervallo temporale.

La persistenza segue ownership esplicita per contesto: ogni servizio possiede il proprio schema Mongo e non condivide accesso diretto al database altrui, con l'eccezione di proiezioni di sola lettura necessarie a risoluzione impatti notifiche.

0.3.6 Sicurezza e controllo accessi

La sicurezza applicativa si basa su JWT e distinzione dei privilegi per ruolo.

Le rotte pubbliche sono limitate a registrazione/login/guest. Tutte le altre richiedono token valido. Le operazioni amministrative (catalogo admin, reporting, gestione ordini admin) richiedono ruolo **ADMIN**.

Il gateway elimina eventuali header identità inviati dal client e li reinserisce solo dopo verifica JWT, riducendo il rischio di spoofing.

Il modello di accesso implementato distingue chiaramente:

- il ruolo del servizio di autenticazione;
- l'uso dei token per identificare l'utente lato backend;
- la distinzione tra utente guest, utente autenticato e amministratore;
- il fatto che il gateway rappresenta il primo punto di enforcement delle policy di accesso.

0.3.7 Design Pattern utilizzati

Nel codice sono stati applicati pattern architetturali e applicativi coerenti con la natura distribuita del sistema.

- **Microservices + API Gateway**: il backend è suddiviso in servizi autonomi per bounded context, mentre il gateway funge da entry point unico e centralizza autenticazione, propagazione identità e routing verso i servizi interni.
- **Hexagonal Architecture (Ports & Adapters)**: nei servizi backend la logica applicativa dipende da porte astratte (es. repository e provider), mentre dettagli infrastrutturali (Mongo, HTTP, Redis, Socket.IO) sono implementati come adapter. Questo riduce l'accoppiamento tra dominio e tecnologia.
- **Use Case / Application Service**: le operazioni applicative sono modellate come classi di use case con metodo **execute** (ad esempio creazione sessione chat, invio messaggi, gestione sottoscrizioni, update stato ordini), separando orchestrazione applicativa e trasporto HTTP.
- **Repository Pattern**: l'accesso alla persistenza è incapsulato dietro interfacce repository; le implementazioni concrete Mongo sono sostituibili e testabili in isolamento.
- **Adapter Pattern**: integrazioni esterne sono incapsulate in adapter dedicati. Un esempio rilevante è il chatbot che usa adapter HTTP per recuperare contesto catalogo/configurazione senza dipendere direttamente dai dettagli dei servizi remoti.

- **Event-Driven Integration (Pub/Sub):** la propagazione di variazioni catalogo avviene tramite eventi Redis (es. prezzo/disponibilit ), consumati da Notification e Cart per aggiornamenti asincroni e disaccoppiati.
- **Observer-style su canali realtime:** le componenti realtime basate su Socket.IO seguono un modello publish/subscribe con listener registrati lato client per nuovi messaggi, typing e notifiche push.
- **Facade/BFF lato frontend services:** il frontend espone servizi applicativi (es. `chatbotService`, `catalogService`, `cadService`) come facciata verso API e socket, semplificando l'uso da parte delle view.
- **Composable Pattern (Vue 3):** la logica riusabile di dominio UI (catalogo, carrello, CAD, notifiche) e' estratta in composables, separando presentazione e orchestration.

L'utilizzo di questi pattern ha permesso di avere un sistema modulare e testabile, che permette anche una facile manutenzione e evoluzione. Questo ha permesso inoltre di mantenere single concerns per ogni modulo, riducendo l'accoppiamento tra le componenti.

0.3.8 Organizzazione del frontend

Il frontend e' stato realizzato come una SPA Vue di modo tale da garantire una separazione per responsabilit :

- **views:** schermate applicative (auth, catalogo, CAD, carrello, configurazioni, notifiche, area admin);
- **services:** client HTTP e socket verso i backend services;
- **store:** stato trasversale (sessione, badge carrello, badge notifiche);
- **composables:** logica UI riusabile e orchestration dei flussi.

Le principali viste applicative sono:

- area catalogo e accesso al configuratore;
- configuratore visuale della scaffalatura;
- area carrello e riepilogo richiesta d'ordine;
- area notifiche;
- interfaccia chatbot di supporto;
- area amministrativa per catalogo, gestione stati ordine e reportistica trend ordini.

La navigazione applica guardie di route coerenti con ruoli e tipo utente. In particolare, le viste admin sono riservate a ruolo ADMIN, mentre i flussi utente standard sono disponibili a utenti autenticati, inclusi guest tokenizzati.

0.3.9 Mockup e prototipazione

Da disegnare

0.3.10 Design delle interfacce utente

Il design delle interfacce e' stato orientato a ridurre la complessita' percepita durante la composizione della scaffalatura. Le principali scelte sono state guidate dall'esigenza di mostrare solo le azioni rilevanti, mantenere visibili prezzo e stato della configurazione e fornire supporto contestuale senza interrompere il flusso principale.

Scelte progettuali principali:

- flusso CAD guidato a step (ambiente, categoria, piano colonne, design, finalizzazione);
- persistenza progressiva della configurazione ai passaggi confermati, con ripresa in sessioni successive;
- feedback immediato tramite toast, stati di loading ed error handling coerente;
- notifiche in-app non intrusive con badge e inbox dedicata;
- chatbot integrato nel flusso utente con risposte contestuali su catalogo;
- separazione netta tra viste utente e viste amministrative.

0.3.11 Considerazioni finali di progetto

Nel complesso, il design di KompozeR bilancia semplicita' d'uso lato utente e modularita' lato backend. L'architettura a servizi con gateway centralizzato, il frontend SPA organizzato per responsabilita' e l'uso combinato di REST e canali real-time forniscono una base coerente con gli obiettivi del progetto.

La soluzione e' inoltre estendibile: nuovi moduli applicativi possono essere aggiunti preservando separazione dei contesti, contratti di API e coerenza dei flussi utente.

0.4 Tecnologie

Per il progetto e' stato adottato uno stack di tipo **MEVN** esteso a microservizi (MongoDB, Express, Vue, Node.js), con componenti aggiuntivi per realtime e integrazione eventi.

Le tecnologie utilizzate per sviluppare il sistema sono:

- **Node.js** ed **Express**: sviluppo dei servizi backend e dell'API Gateway;
- **Vue 3**: sviluppo componente client (SPA);
- **MongoDB** + **Mongoose**: persistenza dati per i diversi contesti applicativi;
- **Socket.IO**: comunicazione realtime per notifiche e chatbot;
- **Redis**: broker in-memory per eventi publish/subscribe tra servizi;
- **Docker** + **Docker Compose**: esecuzione e deploy locale dell'intero sistema.

I linguaggi e le tecnologie di sviluppo usate nel progetto sono:

- **TypeScript**: sia lato client sia lato server;
- **HTML**: struttura delle pagine tramite template Vue;
- **CSS**: definizione stile e layout delle interfacce.

0.4.1 Docker

Docker rappresenta lo standard de-facto per la distribuzione di sistemi multi-componente. Nel progetto KompozeR l'applicazione viene avviata tramite **docker-compose**, che orchestra frontend, API Gateway, microservizi backend, MongoDB e Redis.

Nel file di compose sono presenti tre gruppi principali di servizi:

- **infrastruttura**: **redis** e istanze Mongo dedicate (auth, catalog, cad, cart, order, notification, chat);
- **backend**: **auth-service**, **catalog-service**, **cad-service**, **cart-service**, **order-service**, **notification-service**, **chatbot-service**, **reporting-service**, **api-gateway**;
- **frontend**: servizio **frontend** (Vite).

Nel caso di KompozeR il ruolo di "entry point" e' svolto da **api-gateway**, che centralizza autenticazione JWT, routing delle chiamate HTTP e gestione degli handshake realtime verso i servizi applicativi.

0.4.2 WebSocket

WebSocket e' la tecnologia che consente comunicazione bidirezionale e persistente tra client e server. Nel progetto e' implementata tramite Socket.IO per due funzionalita' centrali:

- **notifiche realtime:** push immediato di eventi utente (es. variazioni di disponibilita'/prezzo rilevanti);
- **chatbot:** scambio messaggi utente-bot e indicatore di typing in tempo quasi reale.

Rispetto al polling periodico, questo approccio riduce il numero di richieste inutili e migliora la latenza percepita dall'utente, perche' l'aggiornamento avviene appena l'evento e' disponibile.

0.4.3 Redis

Redis e' usato come componente in-memory per la comunicazione asincrona tra servizi (pub/sub), in particolare per propagare eventi di catalogo verso i moduli che devono reagire in modo disaccoppiato (ad esempio notifiche e riallineamento carrello).

0.5 Codice

In questa sezione vengono riportati alcuni frammenti di codice rappresentativi del progetto, con l'obiettivo di mostrare come le scelte architetturali siano state tradotte in implementazione concreta.

0.5.1 Backend

Nel listato seguente e' mostrato uno use case del `chatbotService` (`SendSessionMessage`) che valida l'input, verifica ownership/stato sessione, persiste i messaggi e genera una risposta contestuale usando repository e provider esterni.

```
export class SendSessionMessage {
  constructor(
    private readonly repo: ChatRepository,
    private readonly catalog: CatalogQaProvider,
    private readonly cad?: CadConfigurationProvider,
  ) {}

  async execute(input: SendSessionMessageInput): Promise<SendSessionMessageOutput> {
    if (!input.userId?.trim()) throw new ValidationError('userId is required');
    if (!input.sessionId?.trim()) throw new ValidationError('sessionId is required');
    if (!input.content?.trim()) throw new ValidationError('content is required');

    const session = await this.repo.findSessionById(input.sessionId);
    if (!session) throw new SessionNotFoundError(input.sessionId);
    if (session.userId !== input.userId) throw new ForbiddenError(...);
    if (session.status === 'CLOSED') throw new SessionClosedError(input.sessionId);

    await this.repo.saveMessage(userMessage);
    const answer = await this.generateAnswer(input.content, session.userId, session.configuration);
    await this.repo.saveMessage(botMessage);
    await this.repo.updateSession({ ...session, updatedAt: new Date() });

    return { sessionId: input.sessionId, userMessage: ..., botMessage: ... };
  }
}
```

Questo approccio evidenzia la separazione tra orchestrazione applicativa e dettagli infrastrutturali: lo use case non conosce MongoDB o HTTP in modo diretto, ma utilizza porte astratte. Allo stesso modo, di seguito viene mostrato il repository pattern: prima l'interfaccia di dominio, poi l'adapter Mongo concreto.

```
export interface ChatRepository {
  createSession(session: ChatSession): Promise<void>;
  findSessionById(sessionId: string): Promise<ChatSession | null>;
  updateSession(session: ChatSession): Promise<void>;
}
```

```

    saveMessage(message: ChatMessage): Promise<void>;
    listMessagesBySessionId(sessionId: string): Promise<ChatMessage[]>;
  }

export class MongoChatRepository implements ChatRepository {
  async createSession(session: ChatSession): Promise<void> {
    await ChatSessionModel.create({ ... });
  }

  async findSessionById(sessionId: string): Promise<ChatSession | null> {
    const doc = await ChatSessionModel.findById(sessionId).lean();
    if (!doc) return null;
    return { id: doc._id, userId: doc.userId, ... };
  }

  async saveMessage(message: ChatMessage): Promise<void> {
    await ChatMessageModel.create({ ... });
  }
}

```

La combinazione port+adapter permette di testare gli use cases con fake repository e di sostituire facilmente la tecnologia di persistenza senza modificare il dominio.

0.5.2 Frontend

Per il frontend sono riportati due aspetti centrali: gestione errori di autenticazione e notifiche realtime.

Quando l'utente invia il form di login, il componente `AuthView` richiama lo store di autenticazione, mappa gli errori API in messaggi utente e gestisce la navigazione in caso di successo.

```

async function handleLogin(): Promise<void> {
  error.value = '';
  loading.value = true;
  try {
    await auth.login(login.username, login.password);
    await router.push({ name: auth.homeRouteName });
  } catch (e) {
    error.value = mapLoginError(e);
  } finally {
    loading.value = false;
  }
}

function mapLoginError(err: unknown): string {

```

```

    if (!(err instanceof ApiError)) return 'Errore di accesso';
    if (err.code === 'RESOURCE_NOT_FOUND') return 'Utente Non Trovato';
    if (err.code === 'INVALID_PASSWORD') return 'Password Errata';
    return 'Invalid username or password';
  }

```

Nel template la gestione errore e' visualizzata in modo diretto con binding reattivo:

```

<p v-if="error" class="auth-error" role="alert" aria-live="assertive">
  {{ error }}
</p>

```

Per le notifiche realtime, il servizio websocket apre la connessione solo quando token e identita' sono validi e registra listener per push e riconnessione.

```

this.socket = io('/', {
  path: '/api/ws/notifications/socket.io',
  transports: ['websocket', 'polling'],
  auth: { userId },
  query: { token },
});

onPush(handler: (payload: NotificationPushPayload) => void): () => void {
  this.connect();
  this.socket?.on('notification:push', handler);
  return () => this.socket?.off('notification:push', handler);
}

```

Nel componente root (**App.vue**) il client si sottoscrive ai push e aggiorna store/badge/toast in tempo reale:

```

removeNotificationPushListener = notificationSocket.onPush((payload) => {
  const pushed = payload.data?.notification;
  if (!pushed) return;

  notifications.applyRealtimePush(notification);
  notifications.addToast('warning', notification.message);

  if (notification.type === 'AVAILABILITY_CHANGED') {
    void cart.refreshItemCount();
  }
});

```

Questo flusso elimina polling esplicito lato pagina e mantiene coerenti interfaccia, badge e stato globale quando arrivano eventi dal backend.

0.5.3 Considerazioni finali

I blocchi di codice sopra, mostrano tre aspetti chiave del codice di KompozeR:

- uso sistematico di use case e repository lato backend;
- gestione robusta degli errori lato frontend;
- integrazione realtime basata su socket con aggiornamento immediato della UI.

0.6 Deployment

Per la fase di deployment e' stato fatto uso di **Docker**, scelta che semplifica la distribuzione del sistema e rende l'esecuzione indipendente dalla piattaforma host.

Nel progetto sono separati i due blocchi applicativi principali:

- **frontend**, che espone l'interfaccia utente;
- **backend**, organizzato in microservizi.

La separazione tra frontend e backend e' stata adottata per motivi di modularita' e isolamento: il backend resta riusabile anche da client alternativi (ad esempio mobile), senza dipendere da una singola implementazione UI.

L'interazione tra i componenti e' orchestrata automaticamente tramite **docker-compose**. Oltre ai servizi applicativi, sono presenti i servizi infrastrutturali per la persistenza e la messaggistica.

0.6.1 Servizi in compose

Nel file `docker-compose.dev.yml` sono definiti:

- il **frontend**;
- l'**api-gateway** e i microservizi backend;
- **MongoDB** (istanze dedicate ai contesti applicativi);
- **Redis** per la comunicazione eventi realtime/pub-sub.

Il punto di ingresso unico dell'applicativo è l'**api-gateway**, che instrada le richieste verso i servizi interni.

0.6.2 Avvio locale

L'avvio in locale è intenzionalmente semplice e ripetibile:

- backend + infrastruttura: `docker compose -f docker-compose.dev.yml up --build`;
- frontend in sviluppo: `npm run dev:frontend`.

0.7 Conclusioni

KompozeR e' stato concepito come un'estensione evoluta dell'e-commerce Kompo, con l'obiettivo di rendere piu' semplice e guidata la configurazione e l'acquisto di scaffalature personalizzabili. Il progetto affronta il problema sia dal punto di vista dell'esperienza utente, introducendo un configuratore visuale, un chatbot di supporto e notifiche contestuali, sia dal punto di vista dell'organizzazione software, tramite una struttura modulare basata su servizi distinti.

0.7.1 Risultati raggiunti

I principali risultati ottenuti sono:

- definizione dei requisiti e del dominio applicativo;
- progettazione di un'architettura web coerente con il problema affrontato;
- individuazione di servizi separati per autenticazione, catalogo, configurazione, carrello, notifiche, chatbot e reporting;
- implementazione di un API Gateway con autenticazione JWT centralizzata e propagazione identita' verso i servizi;
- realizzazione del flusso CAD completo con finalizzazione e sincronizzazione automatica del carrello;
- implementazione di notifiche cross-service e reportistica trend ordini in area amministrativa;
- definizione di contratti applicativi chiari per API REST, messaggi realtime e DTO di dominio.

0.7.2 Limiti attuali

I limiti principali della versione attuale sono:

- assenza di una campagna formale di user testing su campione esteso;
- metriche prestazionali e di observability non ancora consolidate in modo sistematico nel report;
- alcuni aspetti di hardening operativo (pipeline CI/CD complete, policy di deploy multi-ambiente).

0.7.3 Sviluppi futuri

Le evoluzioni naturali del progetto includono:

- estensione della copertura test automatizzata e introduzione di metriche di quality gate piu' stringenti;

- miglioramento UX del configuratore e dei meccanismi di feedback in tempo reale (con un'interfaccia più competitiva anche esteticamente);
- consolidamento del deployment verso ambienti diversi da quello locale;
- introduzione di un LLM per migliorare l'interazione con il chatbot e fornire suggerimenti più intelligenti agli utenti.